

Sprint 05

1. Identificacao

- Numero da sprint: 5
- Período: 09/05/2026 - 16/05/2026
- Data da entrega: 16/05/2026
- Proprietário do produto: Gustavo Dantas
- Scrum Master: Carolina Ude
- Time de desenvolvimento: Angelo Alvarenga, Pedro Martins, Thales Maia

2. Objetivo da sprint

Identificar e aplicar padrões de projeto pertinentes à solução ExtraUFLA, priorizando padrões com evidência concreta no código sobre padrões puramente especulativos. A meta inclui um refactor mínimo do servidor para demonstrar *Service Layer* e *Layered Architecture por feature*, mantendo a aplicação enxuta e evitando *overengineering*.

3. Itens do Sprint Backlog

ID	Item	Responsável	Status
SB01	Analisar problemas recorrentes de projeto na proposta	Equipe	☐ Concluído
SB02	Selecionar padrões pertinentes (aplicados e avaliados)	Equipe	☐ Concluído
SB03	Refatorar apps/server introduzindo modules/course/ com con-	Equipe	☐ Concluído
SB04	troller/service/model Expor GET /courses via controller (evidência do <i>Service Layer</i>)	Equipe	☐ Concluído
SB05	Preencher docs/padroes/padroes-de-projeto.md com padrões aplicados, avaliados, alternativas e benefícios	Gustavo Dantas	☐ Concluído
SB06	Preencher docs/07_padroes_de_projeto.md (5 seções)	Gustavo Dantas	☐ Concluído
SB07	Redigir relatório docs/sprints/sprint-05.md	Gustavo Dantas	☐ Concluído

ID	Item	Responsável	Status
SB08	Atualizar entregas incrementais e autoavaliação	Equipe	☐ Concluído

4. Relacao com o conteudo da disciplina

Esta sprint está diretamente ligada ao tópico de **Padrões de Projeto**, com aplicação prática de:

- identificação de problemas recorrentes de projeto em uma base real;
- seleção de padrões pertinentes (GoF e arquiteturais);
- justificativa técnica da escolha, com evidência de código;
- análise de alternativas e adiamento explícito de padrões sem caso concreto;
- relação entre os padrões adotados e os requisitos do produto (RF01-RF15).

Os padrões aplicados — Service Layer, Layered Architecture por feature, Factory Method e Facade — foram escolhidos pela presença efetiva de problema que resolvem hoje, não por catalogação genérica.

5. Artefatos produzidos

- docs/padroes/padroes-de-projeto.md — documento principal da sprint com problemas recorrentes, padrões selecionados (aplicados e adiados), alternativas, rastreabilidade RF↔padrão e conclusão.
- docs/07_padroes_de_projeto.md — versão sintética em 5 seções (tabela, exemplo, alternativas, conclusão).
- apps/server/src/modules/course/course.model.ts — tipos derivados do schema.
- apps/server/src/modules/course/course.service.ts — listCourses e seed-Courses.
- apps/server/src/modules/course/course.controller.ts — Router Express com GET /courses.
- apps/server/src/index.ts — slim bootstrap (Express, CORS, seed inicial, montagem do controller).
- docs/sprints/sprint-05.md — este relatório.
- docs/09_entregas_incrementais.md — cronograma e resumo da Sprint 5 atualizados.
- rubrica/autoavaliacao-entregas.md — seção e tabela-resumo da Sprint 5 atualizadas.

6. Evidencias no GitHub

- Arquivos criados/atualizados nesta sprint:
 - apps/server/src/modules/course/course.model.ts (novo)
 - apps/server/src/modules/course/course.service.ts (novo)
 - apps/server/src/modules/course/course.controller.ts (novo)
 - apps/server/src/index.ts (refatorado)
 - docs/padroes/padroes-de-projeto.md
 - docs/07_padroes_de_projeto.md
 - docs/sprints/sprint-05.md
 - docs/09_entregas_incrementais.md
 - rubrica/autoavaliacao-entregas.md
- Commits relevantes: histórico da branch de Sprint 5 (refactor apps/server + documentação de padrões).
- Issues GitHub vinculadas: #42 (relatório), #44 (PDF de entrega).

- Tag da sprint: sprint-5 (a ser criada na publicação).

7. Evolucao da aplicacao web

Nesta sprint a evolução foi dupla: documental (padrões de projeto registrados com justificativa) e estrutural (refactor mínimo de apps/server).

- Antes: apps/server/src/index.ts concentrava bootstrap, CORS, rotas e a função seed-Courses.
- Depois: apps/server/src/modules/course/ agrega controller, service e model. O index.ts ficou responsável apenas por bootstrap e pela leitura do courses.json no startup (passando os dados para o service).
- Novo endpoint: GET /courses retorna a lista de cursos persistida.
- Compilação validada com `bun run check-types` e `bun run --filter server build (tsdown)`.

O estado funcional continua com RF01 e RF02 implementados. RF03 ganhou uma base de backend (endpoint disponível) — UI será implementada nas próximas sprints.

8. Dificuldades encontradas

- **Risco de *overspecification*.** Tendência inicial foi propor 5-6 padrões GoF “porque ficaria mais completo”. A correção foi listar apenas padrões com evidência real no código e justificar o adiamento explícito dos demais.
- **Diferença entre dev e build no tsdown.** A função de seed original usava `import.meta.url` para localizar data/courses.json; em dev (tsx) e em prod (bundle dist/index.mjs) a árvore de diretórios muda. A solução foi manter a resolução de caminho no index.ts (entry point) e passar os dados já parseados para o service, mantendo este isolado de I/O de filesystem.
- **TypeScript composite + Express.** O retorno de Router() precisou de anotação de tipo explícita para o build composite (tsc -b) — a inferência no controller falhava com TS2883 (caminho não portátil).

9. Revisao do incremento

- O que foi concluído:
 - Documento detalhado de padrões em docs/padroes/padroes-de-projeto.md com 4 padrões aplicados (Service Layer, Layered Architecture, Factory Method, Facade) e 2 avaliados/adiados (Repository, Strategy).
 - Documento síntese em docs/07_padroes_de_projeto.md com tabela, exemplo, alternativas e conclusão.
 - Refactor de apps/server com módulo course/ (model + service + controller) e index.ts enxuto.
 - Endpoint GET /courses funcional.
 - Atualização dos documentos de entregas incrementais e autoavaliação.
- O que ficou pendente:
 - Aplicar a mesma estrutura *package by feature* no frontend (apps/web/src/features/) — prevista para quando os RFs de catálogo forem implementados.
 - Implementação dos RFs ainda planejados (catálogo, detalhes, filtros, busca, gestão de organizações/processos).

10. Pendências para a próxima sprint

- Consolidar a **arquitetura** completa (Sprint 6), formalizando camadas, componentes, dependências e relação com requisitos não funcionais (RNF01-RNF07).
- Estender a organização *package by feature* para apps/web quando o primeiro fluxo de UI consumir o endpoint GET /courses.
- Avaliar quando introduzir o padrão **Repository**: assim que surgirem consultas compostas (filtros, busca) nos RFs de catálogo (RF05-RF07).
- Revisar riscos técnicos após avanço da implementação.

11. Quadro Kanban

A fazer	Em andamento	Concluído
RF05-RF07 (catálogo, filtros, busca)	—	SB01 Análise de problemas recorrentes
RF11-RF12 (processo seletivo)	—	SB02 Seleção de padrões
Estender <i>package by feature</i> no frontend	—	SB03 Refactor modules/course/
Repository quando consultas compostas chegarem	—	SB04 Endpoint GET /courses
Strategy quando ranqueamento de candidatos for definido	—	SB05 padroes-de-projeto.md
—	—	SB06 07_padroes_de_projeto.md
—	—	SB07 sprint-05.md
—	—	SB08 Entregas incrementais + autoavaliação